

W1 Mental Computation

Thursday, September 24, 2026

READINGS Craik, *The Nature of Explanation* (1943), ch. 5
Turing, “Computing Machinery and Intelligence” (1950)

CONTENTS	1	Historical setting	1
	2	Craik and Computationalism	2
	3	Computers and computation	3
	4	Computational Theory of Mind	7
	5	Turing and the possibility of AI	7
	6	AI and CTM: the Physical Symbol System Hypothesis	8

1 Historical setting

- **19th-century psychology.** Psychology becomes a laboratory science. Helmholtz, Pavlov, Freud. Recognition of inner processes, but no mechanical model of what they could be.
- **Behaviorism (early 20th c.).** Empiricist movement in psychology: psychology should predict and control observable behavior alone; inner states are either irrelevant or don’t exist at all. Skinner et al.
- **Cybernetics (mid 20th c.).** A machine-oriented way of looking at the body, which ultimately gave way to neural network analysis (and ultimately contemporary AI). In 1943, the same year as Craik’s book, McCulloch & Pitts modeled neurons as logic gates. Brain-oriented, in contrast with the algorithm-orientation of late-20th c. cognitive science.
- **The cognitive revolution (late 20th c.).** Spearheaded by Chomsky, the cognitive revolution rejected behaviorism, embraced inner states, and pursued the idea that cognition could be understood in computational (and algorithmic) terms.

- **The rise of cognitive science (20th into 21st c.).** Following the cognitive revolution, the consolidation of an interdisciplinary field (psychology, linguistics, computer science, neuroscience, philosophy). Its unifying working hypothesis is CTM, a conjecture rather than a theoretical conclusion. Perceptual science remains the largest block.
- **Today's readings: Craik (1943) and Turing (1950).** Both anticipate the computational theory of mind (and AI), just before the cognitive revolution. First visions of the coming wave.

2 Craik and Computationalism

- **Kenneth Craik (1914–1945).** An early cyberneticist. This piece is the earliest clear statement of computationalism, and its guiding motivations (that I know of).
- **Thought as prediction.** Thought's most fundamental power is predicting events. We design a bridge in our heads instead of building it and seeing whether it collapses (p. 50).
- **Three processes.** (1) "Translation" of an external process into symbols; (2) arrival at other symbols by reasoning or inference; (3) "retranslation" back into external processes, or at least recognizing that the prediction came true (p. 50).
- **The calculating machine as a model of a person.** Calculating machines, anti-aircraft predictors, and Kelvin's tide predictor do what thought does, going through the same three stages (p. 51). Craik proposes that brains use comparable mechanisms (p. 61).
- **Physical tokens.** In the machine, the symbols are physical tokens, such as positions of gears. They are manipulated by mechanical processes and read back out as predictions (p. 51).
- **Models and relation-structure.** A model is any physical system with a "similar relation-structure" to the process it imitates. It needn't resemble it pictorially: Kelvin's tide predictor is pulleys and levers (p. 51).
- **The hypothesis.** "My hypothesis then is that thought models, or parallels, reality—that its essential feature is not 'the mind', 'the self', 'sense-data', nor propositions but symbolism" (p. 57). This symbolism is of the same kind we find in calculating machines.

- **Into the brain.** Translation becomes sensory stimulation of neural patterns; reasoning becomes the interaction of those patterns; retranslation becomes the excitation of the motor organs (p. 53).
- **Mind as input/output machine.** Perception → thought → action. This image of the arc of cognition is as old as Descartes, but remains the dominant picture.

3 Computers and computation

What is a computer?

- **Computers.** A computer is a machine whose function is to process information, that is, to carry out inferential (rational?) transitions between contentful states. Unlike a pasta maker, that function can't be described in purely physical terms.
- **Symbols.** A computer processes information by processing symbols: physical objects or patterns that it takes as input, transforms, and produces as output.
- **Representations.** A representation is a symbol that is *about* something. What it is about is its *content*.
- **Representational systems.** A representational *system* specifies rules that map symbols to contents, and symbols only have content relative to a system. "Uno" means one in Spanish; "101" means five in binary.
- **Algorithms.** An algorithm is a mechanical procedure for manipulating symbols. It has finitely many steps, and each transition between steps is completely determinate. By following one, a computer's symbol transformations mirror rational transitions.
- **Computer architecture.** Following Turing, there are three key components:
 - **Store.** "The store is a store of information, and corresponds to the human computer's paper, whether this is the paper on which he does his calculations or that on which his book of rules is printed" (p. 437).
 - **Executive unit.** "The executive unit is the part which carries out the various individual operations involved in a calculation" (p. 437).
 - **Control.** "It is the duty of the control to see that these instructions are obeyed correctly and in the right order" (p. 437).

Defining computation

- **The core idea.** Computation is a relation between a machine and a function. It holds when the machine is designed to physically mirror, or simulate, the argument–value profile of the function.
- **Computing a function.** M computes f under R iff, for every possible argument x of f , if M takes as input a symbol string that represents x , then M produces as output a symbol string that represents $f(x)$.
- **A three-way relation.** Computation relates a machine M , a representational system R , and a function f . Given any two, you can solve for the third.
- **Independence.** The same machine can compute different functions under different systems, and the same function can be computed by different machines.
- **The Tower Bridge.** In Cummins’s (1991) picture, the top span is the function (arguments $\rightarrow$ value), the bottom span is the machine (inputs $\rightarrow$ output), and the towers are interpretation. Under interpretation, the machine’s transitions are revealed as, e.g., addition.

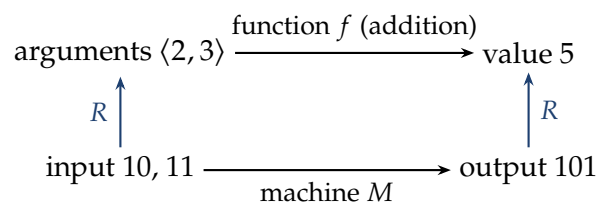


Figure 1. The Tower Bridge. The towers are the interpretation R (here, binary); under it, the machine’s input–output transition mirrors the function’s argument–value pair.

Crash course: numerals, bases, tally and binary

- **Numerals vs. numbers.** Numerals are types of physical symbols; numbers are abstract quantities. The same number has different numerals in different systems: 5, V, 101, 11111.
- **Same numeral, different number.** 11 is three in binary and two in tally.
- **Atomic and complex numerals.** A complex numeral is a string of atomic numerals, each in a position. In Indo-Arabic, the atomic numerals are 0–9.

- **Positional systems.** A positional system is defined by (i) the values of its atomic symbols (starting from zero) and (ii) the values of its positions. Arabic numerals are positional; Roman numerals aren't.
- **Base.** The base is the number of atomic symbols: base 10 has 0–9, base 2 (binary) has 0 and 1, and base 1 (tally) has a single stroke. The base fixes the value of each position.
- **Position values.** Base 10: ones, tens, hundreds, Base 2: ones, twos, fours, eights, sixteens, Tally: every position is a one.
- **Adding up a numeral's value.** Multiply each digit by its position's value, then sum. 111 in decimal = $1 \times 100 + 1 \times 10 + 1 \times 1 =$ one hundred eleven. 111 in binary = $1 \times 4 + 1 \times 2 + 1 \times 1 =$ seven. 111 in tally = $1 + 1 + 1 =$ three.
- **Counting.** Decimal: 1 2 3 4 5. Binary: 1 10 11 100 101. Tally: 1 11 111 1111 11111.
- **Adding in binary.** $0 + 1 = 1$; $1 + 1 = 10$ (write 0, carry 1). So $0111 + 1 = 1000$ (seven plus one is eight): the carry ripples left.
- **Tally conventions.** A well-formed tally numeral is a string of 1s with any number of 0s to its left ($0011 =$ two). 0 counts as the tally numeral for zero.
- **Significance of representational system.** Tally +1 is trivial: add a stroke. Binary +1 needs carrying, but binary is exponentially more compact (about $\log n$ digits instead of n). Choosing the representation changes the machine you need.

Number	Decimal	Binary	Tally
one	1	1	1
two	2	10	11
three	3	11	111
four	4	100	1111
five	5	101	11111
<i>position values</i>	. . . , 100, 10, 1	. . . , 4, 2, 1	. . . , 1, 1, 1

Table 1. Counting to five in three systems.

Finite state automata

- **Discrete states.** A discrete-state machine moves by jumps between definite states. Strictly, nothing physical does; the discreteness is an idealization. As Turing puts

it, “it is a convenient fiction that each switch must be definitely on or definitely off” (1950, p. 439).

- **State.** A state is a system’s overall configuration. Given an input, the state determines the current output and the next state. State abstraction sets aside the circuitry and describes the system only by inputs, outputs, and transitions.
- **FSM diagrams.** Circles are states, and S0 is the initial state. An arrow labeled X:Y means: on input X in this state, output Y and move along the arrow.
- **Example 1: the on/off switch.** Two states, OFF and ON. A PRESS toggles between them; NO PRESS leaves the state as it is. With 1 = press and output = light: S0 (off) 1:1 $\rightarrow$ S1, 0:0 $\rightarrow$ S0; S1 (on) 1:0 $\rightarrow$ S0, 0:1 $\rightarrow$ S1.
- **Example 2: tally successor (+1 T).** S0: 1:1 $\rightarrow$ S0, 0:1 $\rightarrow$ S1. S1: 0:0 $\rightarrow$ S1. Reading right to left, input 0011 (two) gives output 0111 (three): copy the strokes, then add one more.
- **Example 3: binary successor (+1 B).** S0 (carry): 1:0 $\rightarrow$ S0, 0:1 $\rightarrow$ S1. S1 (no carry): 1:1 $\rightarrow$ S1, 0:0 $\rightarrow$ S1. Input 0011 (three) gives output 0100 (four).
- **Same function, different machines.** +1 T and +1 B machines compute the same function under different representational systems. Their physical organization differs; their information-processing function is the same.
- **Limits.** Finitely many states means finite memory. Next week, Chomsky argues that fragments of English conform with a finite state machine, but in general that English *can’t* be generated by a finite state machine.

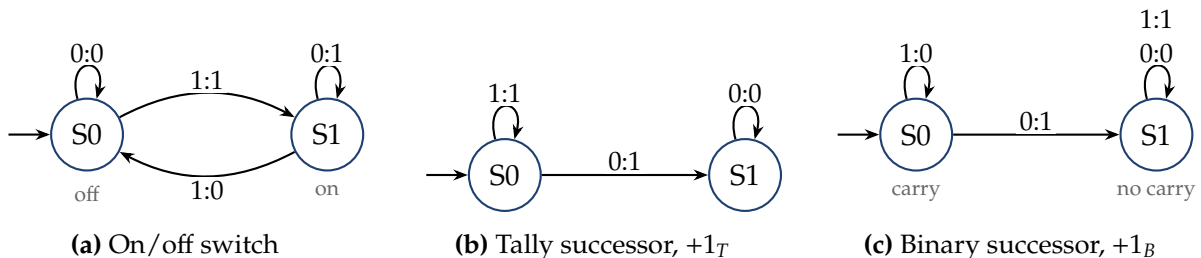


Figure 2. Three finite state machines. An arrow labeled X:Y means: on input X, output Y and follow the arrow. The successor machines read their input right to left.

4 Computational Theory of Mind

- **Not one definition.** Rather than nailing CTM down to one definition, we get a cluster. Some define it in terms of the mind, some in terms of the brain.
- **One version.**
 - (1) The brain is a naturally occurring computer, or information processor.
 - (2) Cognition arises from the computational activity of the brain.
- **With texture.** The brain is a natural computer. Its biological function is to perform a range of cognitive functions by mechanically transforming physical symbols in ways that compute those functions. Cognitive states are computational states of the brain; cognitive processes are computational processes of the brain.
- **Not the computer metaphor.** CTM is not an analogy. It is the computer concept being applied literally to biological agents.
- **Computer or program?** You'll sometimes hear "the mind is the software of the brain"—is this CTM? A program is a set of instructions distinct from the hardware that stores them, but much of the mind emerges from the biological hardware itself. The concept of *algorithm* subsumes both hardware constraints and programs.
- **All of the mind?** Few computationalists think computation explains phenomenology. CTM is really about cognition, broadly construed.

5 Turing and the possibility of AI

- **Turing.** One of the founders of the contemporary algorithm concept, and of computer design in general. "On Computable Numbers" (1936) introduced the Turing machine and the universal machine; after WWII, Turing designed early stored-program computers.
- **The possibility of AI.** Turing maintains both the conceptual possibility and technical plausibility of AI.
- **The Turing Test (imitation game).** "Can machines think?" (p. 433). Turing replaces it with the imitation game. An interrogator questions two hidden players by text and has to tell which is the machine (§§1–2).
 - **Task goal.** Actually, Turing's original version is a bizarre gender-guessing game, not the vanilla "Turing Test" usually associated with him.

- **Linguistic emphasis.** Note that Turing (like Descartes) uses language use as the definitive test of intelligence.
- **Learning machines.** “Instead of trying to produce a programme to simulate the adult mind, why not rather try to produce one which simulates the child’s?” (p. 456).
- **The black box.** “An important feature of a learning machine is that its teacher will often be very largely ignorant of quite what is going on inside” (p. 458).

6 AI and CTM: the Physical Symbol System Hypothesis

- **The AI program.** There is or will be intelligent or mind-like machinery in computers: mind-like, artificially constructed computers.
- **CTM.** Human cognition is realized through computation. The human mind is a computer.
- **Shared presupposition.** In principle, these are not strictly interacting claims. But both presuppose that a physical system, in particular a kind of physical symbol-manipulating system, can give rise to genuine forms of mentality or cognition.
- **Why both belong to cognitive science.** Both draw on this shared presupposition. That presupposition is what we will examine.
- **Opposite ends.** Computer science tries to build unknown artificial intelligence from known physical parts. Cognitive science tries to explain known natural intelligence from unknown physical parts: engineering in reverse.
- **The Physical Symbol System Hypothesis.** “A physical symbol system has the necessary and sufficient means for general intelligent action” (Newell & Simon 1976, p. 116).